

Практика 7. Свёртки

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Структуры данных в лямбда-исчислении, операции с ними
- ▶ Конструкторы данных и паттерн-матчинг в Haskell
- ▶ Классы типов различных кайндов
- + Классы типов `Semigroup`, `Monoid`
- + Класс типов `Foldable`

Содержание

1. Полугруппы и моноиды
2. Свёртки
3. Ленивость и выбор свёртки ($\oplus$)
4. Материалы ()

Содержание

1. Полугруппы и моноиды
2. Свёртки
3. Ленивость и выбор свёртки ($\oplus$)
4. Материалы ()

Полугруппы и моноиды

△ **Полугруппа** — множество с ассоциативной бинарной операцией:

$$(x \langle \rangle y) \langle \rangle z \equiv x \langle \rangle (y \langle \rangle z)$$

```
class Semigroup a where
  (<>) :: a -> a -> a
```

△ **Моноид** — полугруппа с нейтральным элементом: $x \langle \rangle \text{mempty} \equiv \text{mempty} \langle \rangle x \equiv x$

```
class Semigroup a => Monoid a where
  mempty :: a
```

todo Сделайте список полугруппой и моноидом

todo Сделайте статистику результатов тестов полугруппой и моноидом

```
data Counts = Counts { errors :: Int, total :: Int }
```

todo Сделайте эндоморфизм полугруппой и моноидом

```
data Endo a = Endo { getEndo :: a -> a }
```

todo Сделайте произвольную функциональную стрелку полугруппой и моноидом

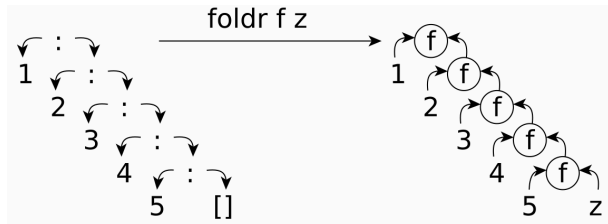
todo Сделайте **Maybe** полугруппой и моноидом

Содержание

1. Полугруппы и моноиды
2. Свёртки
3. Ленивость и выбор свёртки ($\oplus$)
4. Материалы ()

Правая свёртка списка или ...¹

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```



e.g. `foldr f z [a, b, c] ≡ f a (f b (f c z))`

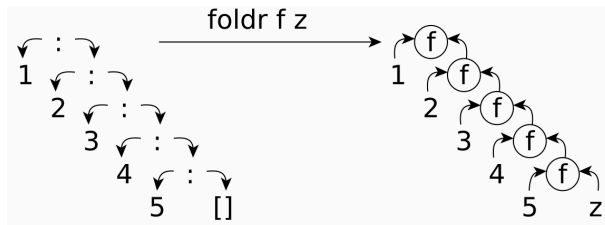
⇒ То есть списки Чёрча — свёртка **data** списков

- ▶ Аналогично и другие структуры данных чистого лямбда-исчисления
- ▶ А с такими структурами можно, как мы убедились, делать всё то же самое

¹За картинки спасибо Михаилу Беляеву.

Задачи на правую свёртку списков

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```



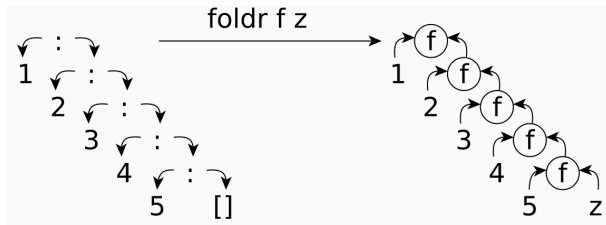
todo Реализуйте скалярное умножение двух векторов, заданных списками

todo Реализуйте группировку пользователей по уровню подписки

```
(Data.Map.insertWith :: (a -> a -> a) -> k -> a -> Map k a -> Map k a)
```

Задачи на правую свёртку списков

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```



todo Реализуйте скалярное умножение двух векторов, заданных списками

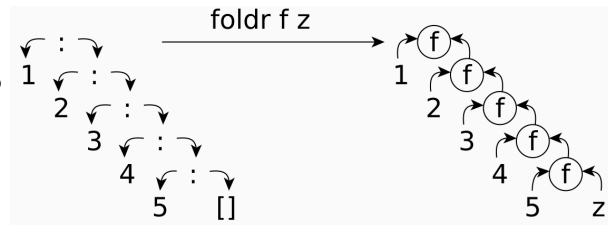
todo Реализуйте группировку пользователей по уровню подписки

```
(Data.Map.insertWith :: (a -> a -> a) -> k -> a -> Map k a -> Map k a)
```

⚡ **prod** xs ys = foldr (+) 0 \$ zipWith (*) xs ys

Задачи на правую свёртку списков

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z [] = z
foldr f z (x:xs) = f x (foldr f z xs)
```



todo Реализуйте скалярное умножение двух векторов, заданных списками

todo Реализуйте группировку пользователей по уровню подписки

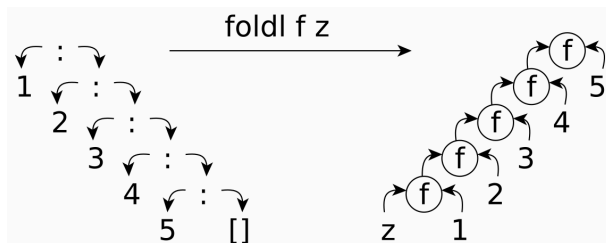
```
(Data.Map.insertWith :: (a -> a -> a) -> k -> a -> Map k a -> Map k a)
```

⚡ `prod xs ys = foldr (+) 0 $ zipWith (*) xs ys`

⚡ `group = foldr (\user -> Map.insertWith (++) (subscr user) [user]) Map.empty`

Левая свёртка списков

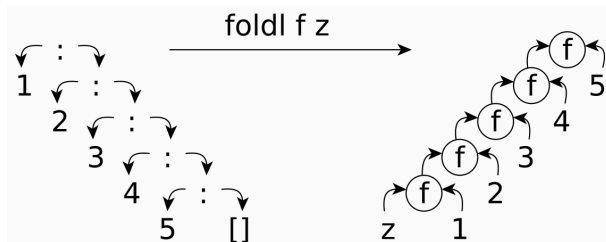
```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z [] = z
foldl f z (x:xs) = foldl f (f z x) xs
```



`todo` Вычитите из чистой прибыли все убытки

Левая свёртка списков

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z [] = z
foldl f z (x:xs) = foldl f (f z x) xs
```



`todo` Вычтите из чистой прибыли все убытки



```
income :: Num a => a -> [a] -> a
```

```
income earned expenses = foldl (-) earned expenses
```

Класс типов `Foldable`

Коллекции, которые можно обойти и аккумулятировать все элементы.

```
class Foldable (f :: Type -> Type) where
  foldr :: (a -> b -> b) -> b -> f a -> b
  foldMap :: Monoid m => (a -> m) -> f a -> m
  ...
```

`todo` Сделайте пару представителем `Foldable`

`todo` Реализуйте `foldMap` через `foldr`

`todo` Реализуйте `foldr` через `foldMap`. Что изменится для `foldl`?

Класс типов Foldable

Коллекции, которые можно обойти и аккумуляровать все элементы.

```
class Foldable (f :: Type -> Type) where
  foldr :: (a -> b -> b) -> b -> f a -> b
  foldMap :: Monoid m => (a -> m) -> f a -> m
  ...
```

`todo` Сделайте пару представителем `Foldable`

`todo` Реализуйте `foldMap` через `foldr`

`todo` Реализуйте `foldr` через `foldMap`. Что изменится для `foldl`?

```
⚡ instance Foldable ((,) e) where
  foldMap :: Monoid m => (a -> m) -> (e, a) -> m
  foldMap f (_, x) = f x
```

Класс типов Foldable

Коллекции, которые можно обойти и аккумуляровать все элементы.

```
class Foldable (f :: Type -> Type) where
  foldr :: (a -> b -> b) -> b -> f a -> b
  foldMap :: Monoid m => (a -> m) -> f a -> m
  ...
```

`todo` Сделайте пару представителем `Foldable`

`todo` Реализуйте `foldMap` через `foldr`

`todo` Реализуйте `foldr` через `foldMap`. Что изменится для `foldl`?

```
⚡ instance Foldable ((,) e) where
  foldMap :: Monoid m => (a -> m) -> (e, a) -> m
  foldMap f (_, x) = f x
```

```
⚡ foldMap f xs = foldr (\x acc -> f x <> acc) mempty xs
```

Класс типов Foldable

Коллекции, которые можно обойти и аккумуляровать все элементы.

```
class Foldable (f :: Type -> Type) where
  foldr :: (a -> b -> b) -> b -> f a -> b
  foldMap :: Monoid m => (a -> m) -> f a -> m
  ...
```

`todo` Сделайте пару представителем `Foldable`

`todo` Реализуйте `foldMap` через `foldr`

`todo` Реализуйте `foldr` через `foldMap`. Что изменится для `foldl`?

```
⚡ instance Foldable ((,) e) where
  foldMap :: Monoid m => (a -> m) -> (e, a) -> m
  foldMap f (_, x) = f x
```

```
⚡ foldMap f xs = foldr (\x acc -> f x <> acc) mempty xs
```

```
⚡ foldr f ini xs = foldMap (Endo . f) xs `appEndo` ini
```

Рецепт приготовления специализированных свёрток

- ▶ Для каждой `data` существует специализированная свёртка
- ▶ Заменяем все конструкторы в дереве на функции соответствующих типов

1. Берём функций по числу конструкторов
2. Типы аргументов — типы аргументов конструкторов
3. Рекурсивные типы — результирующий тип свёртки

```
data List a = Nil | Cons a (List a)
foldList :: List a -> b -> (a -> b -> b) -> b
```

```
todo data Bool = False | True
todo data Nat = Zero | Suc Nat
todo data Pair a b = Pair a b
```

Рецепт приготовления специализированных свёрток

- ▶ Для каждой **data** существует специализированная свёртка
- ▶ Заменяем все конструкторы в дереве на функции соответствующих типов

1. Берём функций по числу конструкторов
2. Типы аргументов — типы аргументов конструкторов
3. Рекурсивные типы — результирующий тип свёртки

```
data List a = Nil | Cons a (List a)
foldList :: List a -> b -> (a -> b -> b) -> b
```

```
todo data Bool = False | True
todo data Nat = Zero | Suc Nat
todo data Pair a b = Pair a b
```

```
⚡ bool :: r -> r -> Bool -> r
nat :: r -> (r -> r) -> Nat -> r
uncurry :: (a -> b -> r) -> (a, b) -> r
```

Работа с коллекциями (⚙)

Origami Programming

«Recursive equations are the “assembly language” of functional programming, and direct recursion the go-to» – Jeremy Gibbons, Origami Programming (The Fun of Programming), 2003

- Свёртки наглядно выражают идею аккумуляирования коллекции

```
state = initial_state
for x in xs:
    state = update_state(x, state)
```

- Собираем программы из универсальных кирпичиков:

```
awarded = students -- (&) :: a -> (a -> b) -> b
& map \(Student name marks) -> (name, average marks))
& filter ((> 0) . snd)
& foldMap \(name, s) -> Map.singleton name ("clever cooky: " ++ show s))
```

Содержание

1. Полугруппы и моноиды
2. Свёртки
3. Ленивость и выбор свёртки (⊕)
4. Материалы (☀)

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно?

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуемся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
GHCi> xs = [1, 2 ..]
```

```
GHCi> :sp xs
```

```
take :: Int -> [a] -> [a]
```

```
take 0 xs = []
```

```
take n [] = []
```

```
take n (x:xs) = x : take (n - 1) xs
```

```
length :: [a] -> Int
```

```
length [] = 0
```

```
length (x:xs) = 1 + length xs
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуеся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs
```

```
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуеся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs
```

```
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
GHCi> ghci> take 1 xs
[1]
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуеся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs
```

```
length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
```

```
GHCi> :sp xs
```

```
xs = _
```

```
GHCi> ghci> take 1 xs
```

```
[1]
```

```
GHCi> :sp xs
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуеся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs

length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
GHCi> ghci> take 1 xs
[1]
GHCi> :sp xs
xs = 1 : _
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуемся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (e.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs

length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
GHCi> ghci> take 1 xs
[1]
GHCi> :sp xs
xs = 1 : _
GHCi> ys = [repeat 1, repeat 2]
GHCi> length ys
2
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуется командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (e.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs

length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
GHCi> ghci> take 1 xs
[1]
GHCi> :sp xs
xs = 1 : _
GHCi> ys = [repeat 1, repeat 2]
GHCi> length ys
2
ghci> :sp ys
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуемся командой `:sprint`.

Ленивая семантика вычислений

- ▶ В Haskell выражение не вычисляется пока оно не нужно
- ▶ Что такое нужно? Значение требуется вывести в реальность (е.g. напечатать)²
- ▶ Просмотреть, что действительно вычисляется можно в консоли³

```
take :: Int -> [a] -> [a]
take 0 xs = []
take n [] = []
take n (x:xs) = x : take (n - 1) xs

length :: [a] -> Int
length [] = 0
length (x:xs) = 1 + length xs
```

```
GHCi> xs = [1, 2 ..]
GHCi> :sp xs
xs = _
GHCi> ghci> take 1 xs
[1]
GHCi> :sp xs
xs = 1 : _
GHCi> ys = [repeat 1, repeat 2]
GHCi> length ys
2
ghci> :sp ys
ys = [_,_]
```

²Смешной анекдот: в Haskell любой алгоритм работает за константу (запоминает, что сделать потом).

³Требуется `:set -XMonomorphismRestriction`. Пользуется командой `:sprint`.

Выбор свёртки списка (⊗)

Выбор свёртки списка (⊕)

foldr

+ Работает с бесконечными списками

- ▶ Правый аргумент `f` — результат свёртки на хвосте списка
- ▶ Если им не всегда пользоваться, он не будет вычислен

```
and :: [Bool] -> Bool; and = foldr (&&) True
-- and [True, False, ...] ≡ True && (False && (... && True))
```

— Реализована не через хвостовую рекурсию

foldl

+ Реализуется через хвостовую рекурсию — быстро, может быть компактно

- ▶ Нужно использовать `Data.List.foldl'` — энергичную версию

```
foldl' f z (x:xs) = let acc = f z x in acc `seq` foldl' f acc xs
```

— Не работает с бесконечными списками

- ▶ Реализация всегда делает рекурсивный вызов на хвосте вне зависимости от `f`

Содержание

1. Полугруппы и моноиды
2. Свёртки
3. Ленивость и выбор свёртки (⊕)
4. Материалы (☀)

Абстракция: цена и новые возможности

- ▶ В других языках есть всё те же примеры полугрупп, моноидов и т.д., но они не обобщены (как правило)
- Мозгу тяжело в абстракции без подготовки и тренировки (и всё равно тяжело)
- Пользователей (программистов) пугают незнакомые слова
- + Можно писать обобщённый код:
`foldMap :: (Monoid m, Foldable f) => (a -> m) -> f a -> m`
- + Обобщённые алгебраические законы гарантируют корректность рефакторинга:

```
fold
    fold ≡ foldMap id
foldMap
    foldMap f ≡ foldr (mappend . f) mempty
```

- + Тестирование выполнения реализацией законов можно абстрагировать в библиотеку:
`import Test.QuickCheck.Classes.Base`
`test = lawsCheck $ foldableLaws $ Proxy @Tree`
- + Намного проще понять новое, увидев в этом лишь пример знакомой общей концепции
- + Компилятор имеет богатые возможности по генерации инстансов стандартных классов

Что посмотреть в транспорте

- ▶ https://en.wikipedia.org/wiki/Deforestation_%28computer_science%29
- ▶ Лекция Дениса Москвина о функциональном программировании (алгебраические рассуждения и оптимизации работы со списками, fusion)
- ▶ Laziness in Haskell video
- ▶ John Hughes - Keynote: How to specify it!

- ▶ build/foldr list fusion
- ▶ Компилятор GHC языка Haskell: теория языков программирования в работе
- ▶ stgi: Educational implementation of the STG (Spineless Tagless G-machine)
- ▶ GHC illustrated
- ▶ GHC Commentary: The Compiler
- ▶ GHC Commentary: The Compiler: generated code